TOS Blockchain

Post-Quantum Authentication and Shielded Transfers

TOS contributors

October 2026

Abstract

TOS is a Layer-1 blockchain with masterchain and shardchain infrastructure, asynchronous smart contracts, and a native virtual machine. Its canonical genesis configuration uses ML-DSA-44 validator signatures. Contracts can verify four post-quantum signature suites: ML-DSA-44, Falcon-512, SLH-DSA-SHA2-128s, and an LMS fee profile. The post-quantum wallets use them for daily authorization, recovery, and fee payment. Its shielded native-asset design combines zero-knowledge transaction proofs, one-time post-quantum note authorization, and post-quantum encrypted note delivery. These mechanisms protect different surfaces: signature security is not the same property as transaction privacy or proof soundness. In particular, the current shielded circuit uses pairing-based Groth16 proofs and is not a post-quantum proof system. This overview describes implementation boundaries, public information, and deployment prerequisites rather than asserting unconditional anonymity or production readiness.

Scope. Source baseline: `a57b8c279`. This document describes the repository's October 2026 implementation and canonical configuration; it is not a statement that every running network has activated them. Exact wire formats and deployment identities remain defined by source and the frozen shielded-pool profile.

Contents

1 Architecture and security objectives **2**

2 Post-quantum consensus from genesis **2**

3 Post-quantum signature suites and wallets **3**

4 Shielded native-asset transfers **4**

5 Cryptographic and financial boundaries **5**

6 Implementation and deployment status **6**

7 Development and operations **6**

Implementation references **6**

1 Architecture and security objectives

The masterchain carries network configuration and validator-set information. Shardchains execute accounts and transfer messages. The TOS Virtual Machine (TVM) executes contracts, while validators authenticate consensus messages. Contract calls between accounts are asynchronous messages: receiving funds, forwarding them, and processing a later result are separate transactions. A signature cannot by itself guarantee completion or refund of that sequence.

TOS applies cryptographic mechanisms at distinct layers:

Surface	Mechanism and boundary
Validator signing	ML-DSA-44; protects authorized consensus messages.
Contract authorization	Native verification of four post-quantum suites, with contract-specific signed fields and replay rules.
Wallet authorization	ML-DSA-44 primary key, SLH-DSA-SHA2-128s protection and recovery key, LMS fee authorization without asset authority.
Shielded note spending	One-time ML-DSA-44 note authorization plus a circuit proof of valid state transition.
Note delivery	ML-KEM-768 key encapsulation and XChaCha20-Poly1305 authenticated encryption.
Hidden-value validity	Groth16 over BLS12-381, with Poseidon2 commitments and tree hashing; not post-quantum proof soundness.

The objective is to provide post-quantum authentication on designated paths and confidentiality within the shielded protocol. It is not to imply that all legacy wallets, administrative credentials, transport authentication, or external applications become post-quantum merely by connecting to TOS.

2 Post-quantum consensus from genesis

2.1 Canonical version parameters

The canonical zerostate generator selects global protocol version 16 in ConfigParam 8, and creates four equally weighted post-quantum genesis validator descriptors. These use `validator_pq_addr#b3` and algorithm identifier 1 for ML-DSA-44. ConfigParam 47 binds controller admission to the approved controller code. The Simplex transport configuration selects its v2 QUIC profile; that transport version is distinct from global protocol version 16.

The public bootstrap manifest is `validator-pq.pub`. It binds each validator's controller identity, ADNL identity, and consensus public key. The canonical generator rejects the classical bootstrap-file format and checks duplicate identities. Private keys remain operator material and must not be placed in the public manifest or committed to the repository. Launching a new chain still requires review of all immutable genesis inputs, including network identity, time parameters, and compiled contract hashes. Changing the generator does not change an existing chain's zerostate.

2.2 Keys and authorization

ML-DSA is standardized in FIPS 204 [1]. The implemented ML-DSA-44 suite uses a 1,312-byte public key and a 2,420-byte signature. Protocol-specific message encodings and domain

separation remain essential: algorithm security does not make a signature valid for another role, network, or request.

A stable controller identity separates validator identity from individual keys. Root authority and consensus signing have different responsibilities; ADNL identity is also distinct. Rotation and admission follow the controller and network rules, rather than an assumption that every address or key is interchangeable. Operators must provision, protect, and recover the correct key for each role.

Validator consensus keys use ML-DSA-44 only. The other suites described in Section 3 are available to contracts and wallets, not to consensus. Every post-quantum instruction has minimum global version 16; the canonical genesis profile is version 16. An instruction's introduction version is not the complete version requirement of a contract using additional instructions.

2.3 Resource implications

Post-quantum signatures increase serialized message sizes relative to compact classical signatures. Admission limits, verification budgets, message-size limits, and network capacity must account for those sizes. This overview makes no throughput or fee claim: measured cost depends on the implementation, message path, hardware, and workload. Native contract verification also does not imply that each contract call repeats consensus verification.

3 Post-quantum signature suites and wallets

3.1 Suites and instructions

TVM verifies four signature suites. The generic `PQCHECKSIG_SUITE` instruction takes the suite number with a message, context, signature, and public key; the dedicated `PQCHECKSIG_MLDSA44`, `PQCHECKSIG_FALCON512_PADDED`, and `LMSCHECKFEEHASH` instructions serve fixed formats. Each verifier charges gas for every input byte before verification and refuses an unknown suite or profile.

Suite	Algorithm	Use and status
1	ML-DSA-44 [1]	Validator consensus keys, Quantum wallet primary key, one-time shielded note keys. Canonical.
2	Falcon-512, padded original profile [5]	Falcon wallet authorization module. Experimental; the standardized FN-DSA profile will be separately versioned.
3	SLH-DSA-SHA2-128s [3]	Quantum wallet protection and recovery key. Hash-based, so its security does not rest on lattice assumptions.
4	LMS fee profile [4]	Quantum wallet fee authorization. Stateful and hash-based; it authorizes fees only.

3.2 Quantum wallet

The Quantum wallet is a post-quantum-only account in workchain 0 with the action set of the standard wallet. Its primary ML-DSA-44 key authorizes everyday transfers. A separate SLH-DSA-SHA2-128s key protects the account: it can lock the primary key, rotate it, and recover the wallet if the primary key is lost or compromised. Both keys derive from one recovery backup through separate derivation roles. Fees are paid from a paired fee vault authorized by LMS signatures, which cannot move the wallet's assets.

The two signature families have different failure modes. A break of the lattice assumptions behind ML-DSA does not by itself give an attacker the SLH-DSA recovery key. LMS keys are stateful: each one-time leaf must be used at most once, so the wallet journals its fee-key state durably.

3.3 Status

The wallet contracts, verifiers, and client tooling are in the repository, together with local transaction evidence. Production custody, default activation, and deployment are not approved: the release gates, including independent review, complete conformance vectors, pricing on supported hardware, and a live-network authorization test, remain open. See the Quantum wallet release status.

4 Shielded native-asset transfers

4.1 State and transaction flow

The version-1 shielded pool targets native TOS in workchain 0. It records commitments and nullifiers rather than exposing a transparent account balance for each private note. Commitments bind note contents; nullifiers prevent a consumed note from being spent again. Proof verification checks the specified transition, including the circuit's value and membership constraints.

Transparent TOS deposit $\longrightarrow$ shielded notes
Shielded notes $\longrightarrow$ proof-authorized private transfer $\longrightarrow$ new notes
Shielded notes $\longrightarrow$ public withdrawal message

Each private transaction has a fixed shape of two input slots and three output slots, with real and dummy data handled according to the profile. The circuit uses Groth16 on BLS12-381, and Poseidon2 over its scalar field for commitments and tree operations. Tree parameters, public inputs, proof encoding, and deployment hashes must match the frozen profile exactly.

Note authorization uses one-time ML-DSA-44 keys. Authorization and proof verification serve different purposes: the former authenticates spending intent, while the latter establishes the hidden-state relationships required by the circuit. Neither can substitute for the other.

4.2 Encrypted delivery

Recipients discover and decrypt note payloads using the wallet's viewing material. Delivery combines ML-KEM-768, standardized in FIPS 203 [2], with XChaCha20-Poly1305. A domain-separated derivation binds key material to the execution domain and output slot, with authenticated context preventing payloads from being silently reassigned to another context.

The fixed payload contains a version byte, a 1,088-byte encapsulation ciphertext, and a 144-byte authenticated encrypted body: 1,233 bytes in total. Real and dummy outputs use the same payload size. This removes a simple length distinction, not all statistical or network side channels.

Payment descriptors are single-use. Viewing keys can remain shared within a pool domain; senders who exchange off-chain descriptor information can recognize a reused viewing key. The current profile therefore does not claim unlinkability against such sender collusion.

4.3 What remains public

Shielded transfers do not hide every observable fact. Transparent deposit values, public withdrawal values and recipients, fees, commitments, nullifiers, transaction ordering, and timing remain observable as specified by the protocol. A node operator or network observer may also see traffic metadata. Wallet compromise can reveal note or viewing material.

Privacy claims must consequently identify the observer and the protected information. Hidden note values and ownership relationships are not the same as hiding network origin, deposit history, or every relationship inferable from user behavior. Wallet practices and the size and activity of the privacy set affect practical privacy.

5 Cryptographic and financial boundaries

5.1 Post-quantum scope

The post-quantum signature suites and ML-KEM delivery address quantum threats to their respective authentication and encapsulation tasks. A wallet that keeps a classical key, or an account that never moves to a post-quantum wallet, keeps its classical exposure. Groth16 is a pairing-based proof construction [6]; its soundness assumptions are not post-quantum. Adding a post-quantum signature to a Groth16 transaction does not make the proof system post-quantum.

The frozen profile explicitly defers a post-quantum proof backend. Future migration would require new proof and circuit review, compatible state and wallet handling, and a controlled activation process. It cannot be represented as an already-delivered property of the current pool.

5.2 Funds, fees, and asynchronous outcomes

A valid proof must still be processed by a correctly funded contract. The pool must preserve backing and enforce fee and message rules. Deposit and transfer entry points use funded internal messages; proof acceptance does not authorize arbitrary expenditure of the account's balance.

For withdrawal, the public amount is the gross outgoing message value before recipient computation, not a guarantee of the recipient's net balance increase. Failure and bounce handling must account for the value actually returned. Fees already consumed by execution or delivery cannot be assumed to reappear as refundable principal. This is an asynchronous settlement boundary, independent of whether the authorization is post-quantum.

The v1 profile does not provide arbitrary multi-asset transfers, variable transaction shapes, or unrestricted payload memos. Its state structures and history are bounded according to the profile. These bounds require explicit capacity and recovery testing; the word "shielded" is not an exemption from storage or execution constraints.

6 Implementation and deployment status

The repository contains the pool contract, circuit, wallet implementation, and deployment artifacts. Availability of those files is not activation approval. The frozen profile labels itself ready for coding and requires its acceptance gates before mainnet or testnet activation.

Development proving keys must never protect real funds. The circuit package describes deterministic development setup material whose secret setup randomness is known. Such material allows forged proofs. A production deployment requires the prescribed ceremony and independently verified artifacts, not merely rebuilding the development example.

Before activation, operators and reviewers must establish at least:

1. Agreement among contract code, circuit, public-input encoding, verifier keys, and deployment-profile identities.
2. Completion of the required ceremony and verification of its final artifacts, without substituting development keys.
3. Native execution of valid and invalid paths, including replay, double spending, malformed payloads, insufficient funding, and asynchronous failures.
4. Resource and recovery checks for the chosen network configuration.
5. Explicit network activation approval under the release and upgrade process; no assumption that source availability authorizes deployment.

This is a summary, not a replacement for the profile's acceptance matrix. There is no administrative `SETCODE` upgrade path in the pool design; changes require the specified network upgrade process.

7 Development and operations

The repository supplies the validator engine, clients, TVM, contract tools, and supporting packages. See the build guide for dependencies and target commands. Network deployment settings are maintained outside this repository; local test success is not a production security certification.

Monitoring should distinguish unavailable observations from a healthy node. Process liveness alone does not establish validator participation, progress, or storage health. Cryptographic validation, operational observations, and fund-recovery tests should each report their own boundaries.

This overview is built from `doc/pq.tex` using `bash scripts/build-pq-paper.sh`. The script requires `pdflatex` and writes `doc/pq.pdf` without leaving intermediate files in the source directory. Historical protocol papers remain available separately.

Implementation references

- Canonical genesis generator and validator bootstrap guide.
- Post-quantum consensus types and native ML-DSA verification interface.

- TVM post-quantum instructions, LMS fee verification, and Falcon wallet profile.
- Quantum wallet release status and implementation record.
- Frozen shielded-pool profile: exact encoding, privacy boundaries, and activation gates.
- Circuit package and development-key warning.
- Shielded wallet implementation.
- Ceremony requirements and network upgrade process.

References

[1] NIST. *Module-Lattice-Based Digital Signature Standard*. FIPS 204, 2024. https://doi.org/10.6028/NIST.FIPS.204.

[2] NIST. *Module-Lattice-Based Key-Encapsulation Mechanism Standard*. FIPS 203, 2024. https://doi.org/10.6028/NIST.FIPS.203.

[3] NIST. *Stateless Hash-Based Digital Signature Standard*. FIPS 205, 2024. https://doi.org/10.6028/NIST.FIPS.205.

[4] NIST. *Recommendation for Stateful Hash-Based Signature Schemes*. SP 800-208, 2020. https://doi.org/10.6028/NIST.SP.800-208.

[5] Pierre-Alain Fouque et al. *Falcon: Fast-Fourier Lattice-based Compact Signatures over NTRU*. Specification v1.2, 2020. https://falcon-sign.info/.

[6] Jens Groth. *On the Size of Pairing-Based Non-interactive Arguments*. EUROCRYPT, 2016. https://doi.org/10.1007/978-3-662-49896-5_11.